

P2 Setting up your repository

In modern scientific research and software development, version control systems (VCS) are essential for tracking changes, maintaining project history, and facilitating collaboration. In this laboratory session, you will learn how to use Git and GitHub to manage the documentation for your scientific project. By creating a central repository, your group can work simultaneously on the report while keeping all versions organized and backed up. This practical activity provides a hands-on introduction to standard collaborative workflows used in bioinformatics and computational science.

By the end of this session, your group will have:

- ☑ Git installed and configured on your machines
- ☑ A GitHub account for each member
- ☑ A shared group repository on GitHub
- ☑ A clean folder structure for your scientific report
- ☑ A `report.md` file with the full report skeleton, ready to fill in
- ☑ Your first commits and a live commit history visible on GitHub

Work in groups. One person per group creates the repository and adds the others as collaborators. From that point, everyone works on their own machine and synchronises through GitHub.

How to read this worksheet

Each step shows up to three ways to do the same thing. Use the one you are most comfortable with, they all produce the same result.

Label	What it means
CLI / VSC	Terminal Command line / Visual Source-code editor
GitHub	The GitHub website at github.com

You do **not** need to use all tracks. Pick one and stick with it. The CLI track is shown first because it maps directly to the theory notes; it is always worth knowing what the underlying command is, even if you use a graphical tool day-to-day.

Phase 1 Setup

🕒 ~15 minutes

Goal: Git installed, GitHub account created, identity configured.

Step 1.1 Install Git

```
git --version
```

If you get `command not found` or no output, install Git:

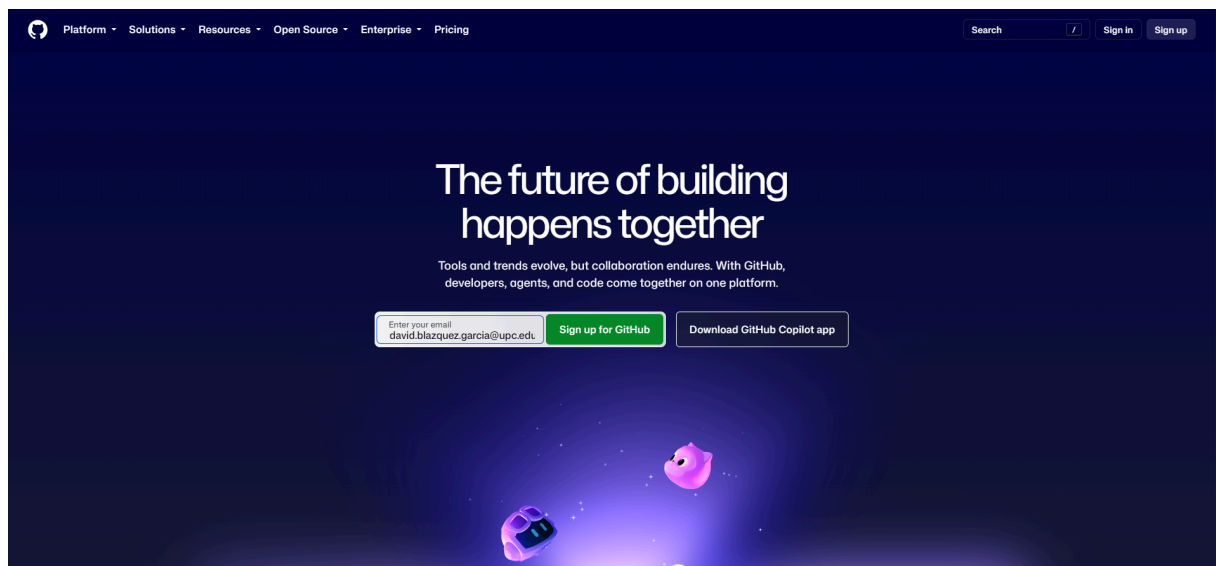
- **Windows:** download from git-scm.com/download/win during installation, choose *Git Bash* and leave all other options as default
- **macOS:** run `xcode-select --install` in the terminal this installs Git as part of the developer tools
- **Linux (Ubuntu/Debian):** `sudo apt install git`

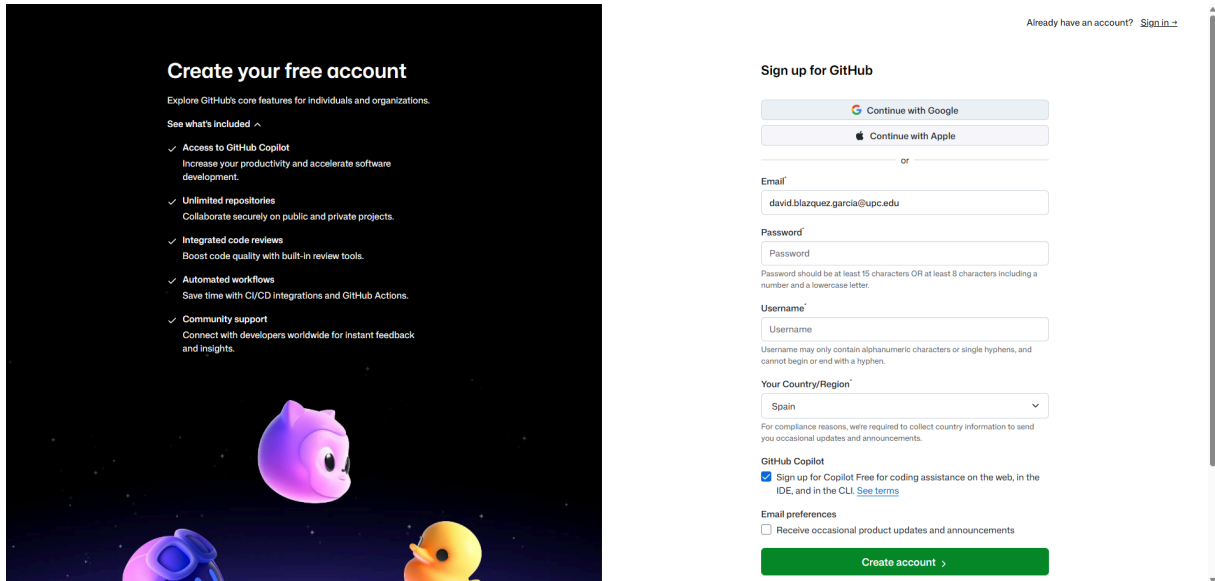
After installation, close and reopen the terminal, then verify:

```
C:\Users\d.blazquez>git --version
git version 2.54.0.windows.1
```

Step 1.2 Create a GitHub account

1. Go to github.com > Type your UPC email > **Sign up for GitHub** > Recommended to Continue with Google





Create your free account

Explore GitHub's core features for individuals and organizations.

See what's included ^

- ✓ **Access to GitHub Copilot**
Increase your productivity and accelerate software development.
- ✓ **Unlimited repositories**
Collaborate securely on public and private projects.
- ✓ **Integrated code reviews**
Boost code quality with built-in review tools.
- ✓ **Automated workflows**
Save time with CI/CD integrations and GitHub Actions.
- ✓ **Community support**
Connect with developers worldwide for instant feedback and insights.

Already have an account? [Sign in](#)

Sign up for GitHub

[Continue with Google](#)

[Continue with Apple](#)

or

Email*
david.blazquez.garcia@upc.edu

Password*
Password
Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username*
Username
Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

Your Country/Region*
Spain

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

GitHub Copilot

☒ Sign up for Copilot Free for coding assistance on the web, in the IDE, and in the CLI. [See terms](#)

Email preferences

☐ Receive occasional product updates and announcements

[Create account](#)

2. Choose a professional username (this will be public, someday you might share it with a reviewer, recruiter or possible manager!)
3. Verify your email in <https://github.com/settings/emails> **required** before you can push commits (If you decided to continue with Google you can skip this step)

Step 1.3 Configure your identity

Git attaches your name and email to every commit you create. This only needs to be done once per machine.

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

Use the same email as your GitHub account.

Checkpoint 1

Everyone on the team should be able to run:

```
git --version    # prints a version
git config user.name  # prints your name
git config user.email # prints your email
```

```
C:\Users\d.blazquez>git config user.name
David

C:\Users\d.blazquez>git config user.email
david.blazquez.garcia@upc.edu
```

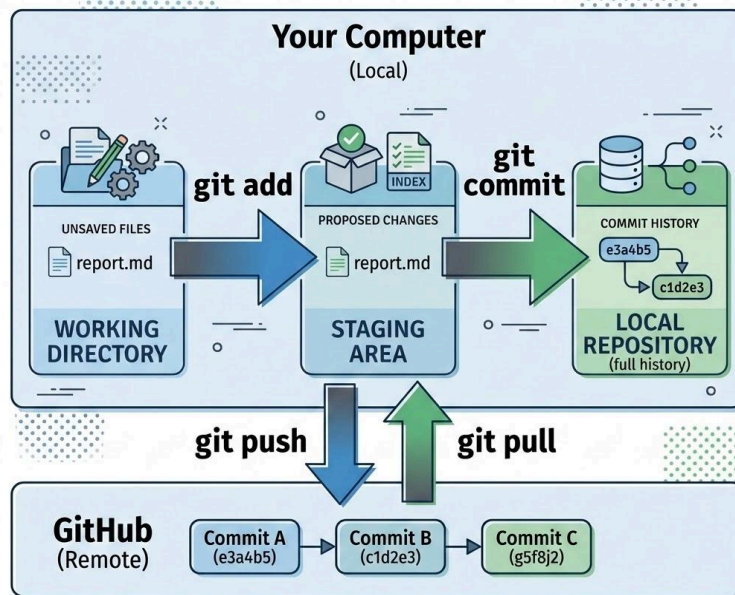
And should be logged into github.com in their browser.

Phase 2 Create the Group Repository

🕒 ~10 minutes

Goal: One shared repository on GitHub, all group members added as collaborators, cloned to everyone's machine.

One person per group does Steps 2.1 and 2.2. Everyone does Step 2.3.

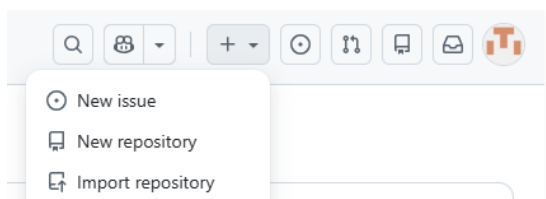


Step 2.1 Create the repository on GitHub

GitHub (Only option)

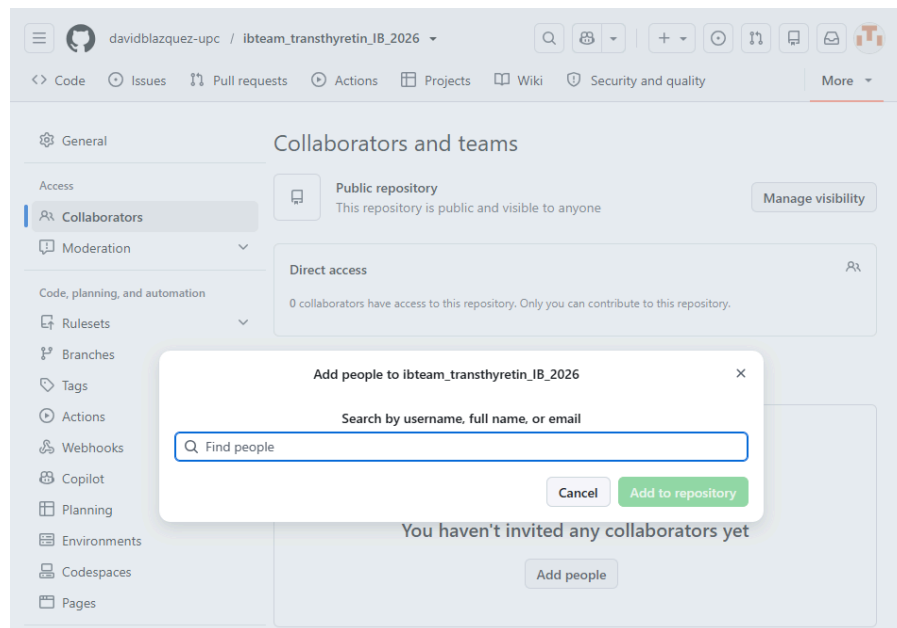
- Log into github.com
- Click + (top-right) → **New repository**
 - Repository name:** `GroupName_DiseaseName_IB_2026` use camel case and underscores
 - Description:** Feel free to populate it with a meaningful text (you can ask Gemini)
 - Visibility:** as instructed (Public or Private)
 - Add a README file:** check this box
 - Add .gitignore:** choose `Python` from the dropdown (useful later when you add scripts)
 - No license**
- Click **Create repository**

You now have a repository with one commit (`Initial commit`) and two files: `README.md` and `.gitignore`.



Step 2.2 Add collaborators

The repository owner goes to Settings (top menu) > Collaborators (left sidebar, under Access) > Add people > Add [username] to this repository



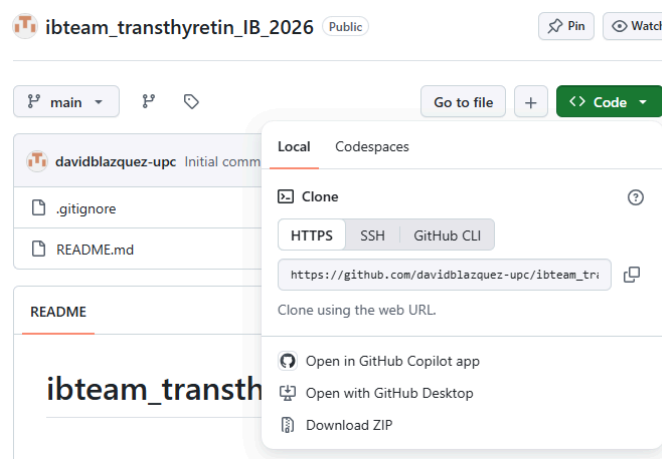
Each member receives an email invitation and must **accept it** before they can push. Tell your teammates to check their email now.

Step 2.3 Clone the repository to your computer

Everyone on the team does this step.

CLI

On the repository page (GitHub), click the green <> **Code** button → **HTTPS** tab → copy the URL



Open your terminal, navigate to where you want to store the project, and run:

```
git clone https://github.com/username/bioinformatics-report-group-a.git
```

```

C:\Users\d.blazquez>cd Projects

C:\Users\d.blazquez\Projects>mkdir GitHub

C:\Users\d.blazquez\Projects>dir
Volume in drive C is Windows
Volume Serial Number is 3891-55F7

Directory of C:\Users\d.blazquez\Projects

16-Sep-26  06:37 PM    <DIR>          .
16-Sep-26  06:36 PM    <DIR>          ..
16-Sep-26  06:37 PM    <DIR>          GitHub

```

```

C:\Users\d.blazquez\Projects\GitHub>git clone https://github.com/davidblazquez-upc/ibteam_transthyretin_IB_2026.git
Cloning into 'ibteam_transthyretin_IB_2026'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.

```

Navigate into the folder:

```
cd bioinformatics-report-group-a
```

GitHub

The web interface cannot clone. You can edit files directly on GitHub in later phases.

Do **NOT** click "Fork": A fork creates a disconnected, separate copy of the repository under your personal account. You will not be able to collaborate directly with your team on the shared repository.

If you plan to work using GitHub's web interface or want to access the project quickly from your profile, click the **Star button** at the top right. This bookmarks the repository under Your stars (Profile → Your stars) so you can find it instantly without needing to save the URL.

Checkpoint 2

Everyone should have a local copy of the repository. Verify:

```
git remote -v
# Should show: origin https://github.com/... (fetch) and (push)
```

```

C:\Users\d.blazquez\Projects\GitHub\ibteam_transthyretin_IB_2026>git remote -v
origin https://github.com/davidblazquez-upc/ibteam_transthyretin_IB_2026.git (fetch)
origin https://github.com/davidblazquez-upc/ibteam_transthyretin_IB_2026.git (push)

```

Phase 3 Build the Repository Structure

🕒 ~10 minutes

Goal: Create the folder structure your report will live in, and make your first commit.

Step 3.1 Create the folder structure

Your repository should have this structure by the end of this step:

```
bioinformatics-report-group-a/
├── .gitignore      (already created by GitHub)
├── README.md       (already created by GitHub)
├── report.md        ← main report document (you will fill this in Phase 4)
├── figures/
│   └── .gitkeep     ← placeholder to track the empty folder
```

Why `.gitkeep`? Git does not track empty folders — it only tracks files. Adding a hidden empty file called `.gitkeep` is the conventional way to include a folder in the repository before you have any real files to put in it.

CLI

```
# Make sure you are inside the repo folder
cd your-folder-path

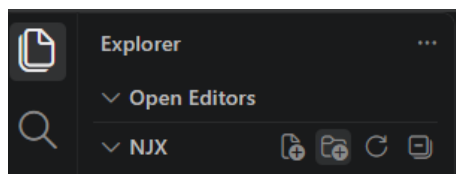
# Create the figures folder and the placeholder file
mkdir figures

# On macOS / Linux / Git Bash:
touch figures/.gitkeep
touch report.md

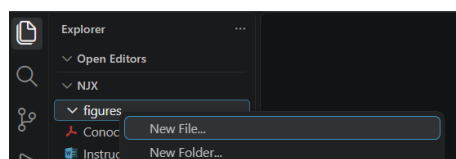
# On Windows (CMD / PowerShell):
type nul > figures\.gitkeep
type nul > report.md
```

VS Code

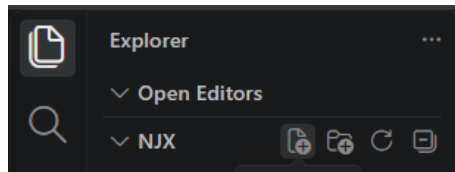
Explorer > **New Folder** icon > name it `figures`



Right-click on `figures` → **New File** → name it `.gitkeep`



Explorer > **New File icon** → name it `report.md`

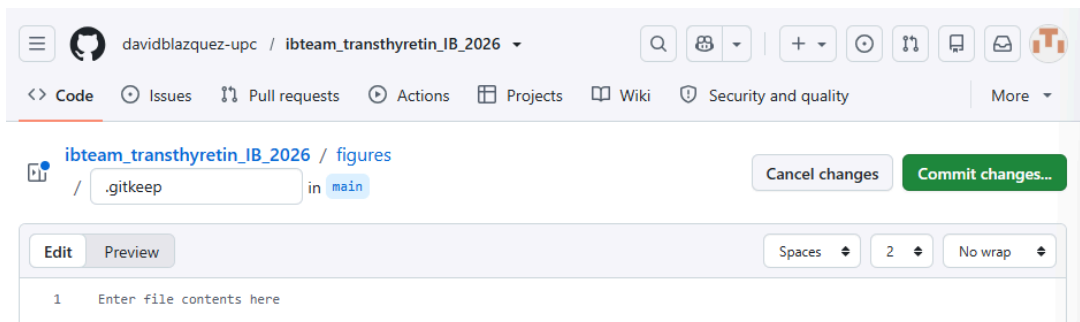


GitHub

Click **Add file > Create new file**

In the name field type: `figures/.gitkeep` (GitHub interprets the `/` as a folder separator and creates the folder automatically). Leave the file content empty

> **Commit changes** Message: `Set up report folder structure`



- ☒ Commit directly to the main branch
- ☐ Create a new branch for this commit and start a pull request [Learn more about pull requests](#)

Cancel

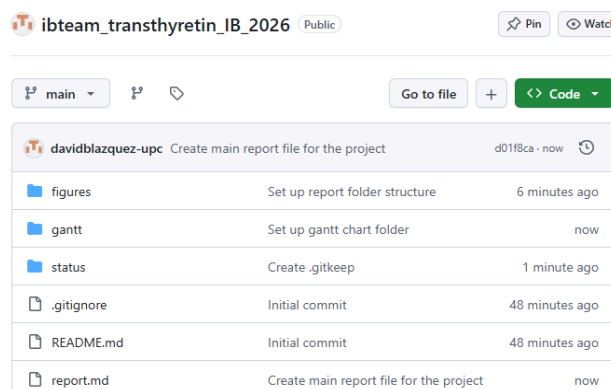
Commit changes

*note we are not using branches for now, you can use main for everything

Repeat: **Add file > Create new file** > `status/.gitkeep` > Commit

Repeat: **Add file > Create new file** > `gantt/.gitkeep` > Commit

Repeat: **Add file > Create new file** > `report.md` (leave empty) > Commit



Step 3.2 Commit the structure

CLI

```
git add .
git status
# Should show: new file: figures/.gitkeep, new file: report.md

git commit -m "Set up report folder structure"
git push
```

Step 3.3 Update the README

The README is the face of your repository. Replace the default content with something informative.

Open `README.md` and replace its contents with:

```
# Bioinformatics Report – [Group Name]

**Course:** Introduction to Bioinformatics · UPC · 2026-2027
**Group:** [Group Name]
**Authors:** [Name 1], [Name 2], [Name 3]...

## Description

*Write a one-paragraph description of your report topic once it is assigned.*

## Repository Structure

- `report.md` – main report document
- `figures/` – images and plots referenced in the report
- `README.md` – this file

## How to Read This Report

Open `report.md` directly on GitHub it renders automatically.
To read it locally, open the file in VS Code or any Markdown viewer.
```

Commit this change with message: `Update README with group info and repo structure`

Checkpoint 3

On GitHub, your repository should now show:

Three files/folders: `README.md`, `.gitignore`, `figures/`, `report.md`

Two or three commits in the history

The README renders correctly as the repository homepage

```
git log --oneline
# Example:
# b7c14e2 Update README with group info and repo structure
# 3f9a001 Set up report folder structure
# c2d8e11 Initial commit
```

Phase 4 Write the Report Skeleton

🕒 ~30 minutes

Goal: Fill `report.md` with the full structure of your scientific report using Markdown. By the end, opening the file on GitHub will show a clean, well-structured document even if most sections only contain placeholder text.

Step 4.1 Open `report.md` in your editor

VS Code

Double-click `report.md` in the Explorer panel. To see the rendered preview side-by-side: right-click the tab Open Preview to the Side (you might need an extension).

CLI + any editor

```
# Open with VS Code from the terminal
code report.md

# Or with nano (terminal editor)
nano report.md
```

GitHub

Click on `report.md` > pencil icon (Edit). You will not see a preview while editing, but you can switch to the **Preview** tab at any time.

Step 4.2 Write the report skeleton

See the [BMC Template](#) for the formal structure of the report we are expecting from you, please follow all the instructions. Today you don't need to have it all set, as said, we are only populating the skeleton. That means, for this step just populate titles and subtitles for all the sections.

Find below an overview of the most used commands in Markdown that will help you with the report.

Markdown Reference Guide

Markdown is a lightweight text format that renders as formatted HTML on GitHub (and most modern editors). Your report lives in a .md file so understanding Markdown is understanding your report's language.

Headings

```
# Level 1 heading – use this for the report title only
## Level 2 heading – main sections (Introduction, Methods...)
### Level 3 heading – subsections
#### Level 4 heading – rarely needed
```

Text emphasis

```
**Bold text** – use for key terms on first use
*Italic text* – use for species names, journal titles, foreign words
~~Strikethrough~~ – for deleted content (useful in revision drafts)
```

Paragraphs and line breaks

In Markdown, a blank line creates a new paragraph. A single newline does **not**.

```
This is the first paragraph.

This is the second paragraph.

This is one sentence.
This continues on the same line when rendered.

This is one sentence.
This is on a new line (note: two spaces at the end of the line above).
```

Lists

Unordered list (use - or *):

```
- First item
- Second item
  - Nested item (indent with two spaces)
  - Another nested item
- Third item
```

Ordered list:

```
1. First step
2. Second step
3. Third step
```

Links and images

```
[Link text] (https://www.example.com)
```

```
[Link with tooltip] (https://www.example.com "Tooltip text")
```

Images follow the same pattern but with a **!** prefix and the path to the file:

```
![Figure 1: PCA plot of sample clusters] (figures/figure1.png)
```

```
![Alt text description] (figures/subfolder/figure2.png)
```

Images in your repo: Store all images in a `figures/` folder. Reference them using the **relative path** from `report.md` this way they display correctly both locally and on GitHub.

Blockquotes

Use for quotes, important notes, or highlighted observations:

```
> This is a blockquote. Use it to highlight a key finding or a quote from a paper.
```

```
> Note: You can use formatting inside blockquotes.
```

Tables

```
| Column A | Column B | Column C |
|---|---|---|
| Row 1, A | Row 1, B | Row 1, C |
| Row 2, A | Row 2, B | Row 2, C |
```

Alignment (optional):

```
| Left | Centre | Right |
|:---|:---:|:---:|
```

Code

Inline code (for commands, file names, short snippets):

```
Run `git status` to check the repository state.
The output file is named `results.csv`.
```python
def saludar():
 print("Hola")
```
```

Horizontal rule

Creates a visual divider — useful between major sections:

```
---
```

Footnotes and references

Markdown does not have a native citation system, but you can use numbered references manually:

```
Sequence alignment was performed using BLAST [1].

---

## References

1. Altschul, S.F. et al. (1990). Basic local alignment search tool. *Journal of Molecular Biology*, 215(3), 403-410.
```

References in your repo: Store all the documents you use in a `references/` folder. Reference them using the **Mini Oxford SCIMED REFERENCE STYLE**.

Step 4.3 Commit the report skeleton

Once you have the skeleton in place:

CLI

```
git add report.md
git commit -m "Add report.md skeleton with full section structure"
git push
```

Step 4.4 Each member adds their name to a section

To practice the collaboration workflow, each group member takes ownership of one section. Edit the `report.md` file to add a line under each section heading indicating who is responsible and what is the estimated extension you want to assign it

```
## Background
*Owner: A. Smith*
Extension 1 page
```

This way you are also starting your project planning, starting to identify tasks, and assign them to an owner.

Collaboration protocol (important to do this in order):

```
# Before you edit: always pull first
git pull

# Make your edit (add your name to your section)
# Then:
git add report.md
git commit -m "Assign Introduction section to Alice Smith"
git push
```

If two people push conflicting edits to the same file, Git will report a conflict. See the "Dealing with conflicts" note at the end of this worksheet.

Remember we want you to collaborate and iterate, not to push one final version into the repo. This means that regular commits are expected from each member of the team!

Checkpoint 4

Open your repository on GitHub. You should see:

- `report.md` with a full section structure rendering correctly
- Each section heading visible in the document
- The commit history showing at least: initial commit → structure → report skeleton → section assignments

Click on any commit hash in the history to see the exact diff the lines added and removed.

Phase 5 Collaboration Practice

🕒 ~15 minutes

Goal: Practice the pull → edit → commit → push cycle as a team, and experience what GitHub's commit history looks like when multiple people contribute.

Step 5.1 Everyone edits the README simultaneously

Each team member:

1. Pulls the latest version: `git pull`
2. Adds one line to the README under a new section `## Team notes` — write something short (your section assignment, a note about the topic, anything)
3. Commits with a meaningful message
4. Pushes

Do this one person at a time the person who pushes second will need to `git pull` before they can push (Git will refuse if your local history is behind).

```
git pull                    # always first
(edit README.md)
git add README.md
git commit -m "Add team notes: Alice assigned to Introduction"
git push
```

This simulates real collaborative work on a shared document.

Step 5.2 Create a branch

Practice creating a branch for a change you are not sure about yet:

CLI

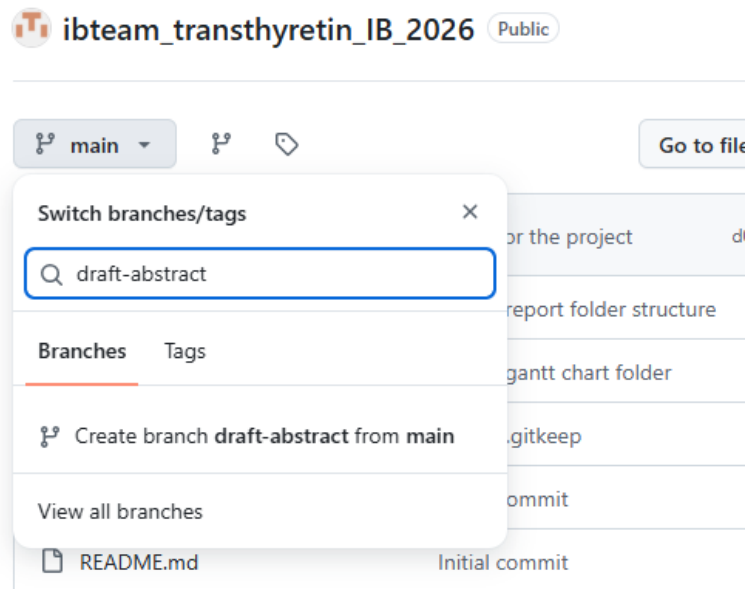
```
# Create and switch to a new branch
git switch -c draft-abstract

# Make a change (write a draft abstract in report.md)
# Then commit it
git add report.md
git commit -m "Draft abstract - to be reviewed"

# Push the branch to GitHub
git push -u origin draft-abstract
```

GitHub (web)

1. Go to your repo → click the branch selector dropdown (`main`)
2. Type `draft-abstract` → click **Create branch: draft-abstract from main**
3. Switch to that branch → edit `report.md` via the pencil icon → commit



Result: the `main` branch is unchanged. The draft lives safely on its own branch. When ready, you can open a Pull Request to propose merging it.

Checkpoint 5

Your repository on GitHub should now have:

- Commits from multiple team members visible in the history
- A meaningful commit log that tells the story of what you built today
- A `draft-abstract` branch visible in the branch selector

Wrap-up

🕒 ~5 minutes

Before you go, map what you did today back to the concepts from the lecture:

| What you did | Git concept |
|----------------------------------|---|
| Created the repo with a README | Initialised a repository with a first commit |
| Cloned the repo to your machine | <code>git clone</code> — copied remote to local |
| Created files and committed them | Working directory → staging → commit |
| Pushed to GitHub | <code>git push</code> — local → remote |
| Pulled before editing | <code>git pull</code> — remote → local (staying synchronised) |
| Wrote commit messages | Annotating your history — the scientific record |
| Viewed the commit history | <code>git log</code> — the timeline of your project |
| Added collaborators | Shared repository — distributed workflow |

Your repository is now live. Every section of the report you write, every figure you add, every revision you make, from today until submission, will be tracked here. Use it. You are free to start working on your report, you can use the papers you found during the previous session to start drafting all the different sections, and please don't leave the lab if you still have doubts. We are here to help you!

Reference — Dealing with Conflicts

When two people edit the same lines of the same file and both push, Git cannot decide which version is correct. It stops and marks the conflict in the file:

```
<<<<<<< HEAD
The sample size was 30 subjects.
=====
The sample size was 32 subjects.
>>>>>>> origin/main
```

To resolve:

4. Open the file — find the conflict markers (<<<<<<<, =====, >>>>>>>)
5. Decide which version is correct (or write a new version that combines both)
6. Delete all three marker lines — keep only the final text
7. Save the file
8. Stage and commit the resolution:

```
git add report.md
git commit -m "Resolve conflict in Methods: confirm n=32 (verified in lab notes)"
git push
```

Conflicts are normal and not a sign that something went wrong. Git is asking you to make a decision that it cannot make automatically. The commit message on a conflict resolution should explain what decision was made and why.

Reference — Daily Workflow (Reminder)

Every time you sit down to work on the report:

```
git pull                                # 1. Get your teammates' latest changes

# (edit your files)

git status                             # 2. Check what you changed
git diff report.md                     # 3. Review line-level changes (optional)
git add report.md                       # 4. Stage
git commit -m "Your message"           # 5. Commit
git push                               # 6. Share
```